`ps`

Muestra los procesos **asociados a la terminal actual.**

- **PID:** identificador único del proceso.
- **TTY:** terminal desde la que se ejecutó el proceso.
- **TIME:** tiempo total de CPU consumido por el proceso.
- **CMD:** comando o programa que inició el proceso.

`ps -e`

Muestra **todos los procesos del sistema**, no solo los asociados a la terminal actual.

- **PID:** identificador del proceso.
- **TTY:** terminal asociada; aparece `?` si no tiene terminal.
- **TIME:** tiempo de CPU consumido.
- **CMD:** comando que inició el proceso.

`ps -ef`

Muestra **todos los procesos**, pero con información más completa.

- **UID:** usuario propietario del proceso.
- **PID:** identificador del proceso.
- **PPID:** identificador del proceso padre.
- **C:** uso reciente de CPU.
- **STIME:** hora o fecha de inicio.
- **TTY:** terminal asociada.
- **TIME:** tiempo total de CPU consumido.
- **CMD:** comando completo con sus argumentos.

`pstree`

Muestra los procesos del sistema en forma de árbol, indicando la relación entre procesos padre e hijos.

Ejemplo:

```
systemd─┬─sshd───bash───pstree
        └─cron
```

Esto significa que:

- `systemd` es el proceso principal.
- `sshd` fue iniciado por `systemd`.

`top`

Muestra en tiempo real los procesos y el uso de recursos del sistema.

```
top
```

Información general de la parte superior

- **up:** tiempo que lleva encendido el sistema.
- **users:** cantidad de usuarios conectados.
- **load average:** carga promedio del sistema en los últimos 1, 5 y 15 minutos.
- **Tasks:** cantidad de procesos totales, ejecutándose, dormidos, detenidos y zombis.
- **%Cpu:** porcentaje de uso de la CPU.
- **MiB Mem:** memoria RAM total, libre, usada y en caché.
- **MiB Swap:** memoria de intercambio total, libre y usada.

Columnas de `top`

- **PID:** identificador único del proceso.
- **USER:** usuario propietario del proceso.
- **PR:** prioridad asignada por el sistema.
- **NI:** valor *nice*, que influye en la prioridad. Cuanto menor, mayor prioridad.
- **VIRT:** memoria virtual total utilizada o reservada.
- **RES:** memoria RAM física utilizada.
- **SHR:** memoria que el proceso comparte con otros.
- **S:** estado del proceso:
  - `R`: ejecutándose.
  - `S`: dormido o esperando.
  - `D`: espera no interrumpible.
  - `T`: detenido.
  - `Z`: zombi.
- **%CPU:** porcentaje de CPU utilizado.
- **%MEM:** porcentaje de memoria RAM utilizado.
- **TIME+:** tiempo total de CPU consumido.
- **COMMAND:** nombre del comando o programa.

`htop`

Es una alternativa más visual e interactiva a `top`.

Instalación

En Ubuntu o Debian:

```
sudo apt install htop
```

Ejecución

htop

Información general de la parte superior

- **CPU:** uso de cada núcleo o procesador lógico.
- **Mem:** cantidad de memoria RAM utilizada.
- **Swp:** cantidad de memoria swap utilizada.
- **Tasks:** cantidad de procesos e hilos.
- **Load average:** carga promedio en 1, 5 y 15 minutos.
- **Uptime:** tiempo que lleva encendido el sistema.

Columnas de `htop`

- **PID:** identificador del proceso.
- **USER:** usuario propietario.
- **PRI:** prioridad actual del proceso.
- **NI:** valor *nice*.
- **VIRT:** memoria virtual utilizada.
- **RES:** memoria RAM física utilizada.
- **SHR:** memoria compartida.
- **S:** estado del proceso.
- **CPU%:** porcentaje de CPU utilizado.
- **MEM%:** porcentaje de RAM utilizado.
- **TIME+:** tiempo total de CPU consumido.
- **Command:** comando completo que inició el proceso.

Ventajas de `htop` frente a `top`

- Tiene una interfaz más clara y con colores.
- Muestra el uso de cada núcleo por separado.
- Permite desplazarse con el teclado o el mouse.
- Permite buscar, ordenar y filtrar procesos fácilmente.
- Permite finalizar un proceso seleccionándolo, sin escribir manualmente su PID.
- Puede mostrar los procesos en forma de árbol.
- Facilita cambiar la prioridad de un proceso.

`top` es más básico y suele venir instalado por defecto. `htop` es más visual, interactivo y fácil de usar.

Memoria en `top` y `htop`

Ambos muestran la misma información básica:

- memoria RAM total, usada y libre;

- memoria destinada a búfer y caché;
- memoria swap;
- consumo de memoria de cada proceso mediante `%MEM`, `VIRT`, `RES` y `SHR`.

La diferencia principal es la presentación:

- **`top`** muestra los valores en texto y de forma más básica.
- **`htop`** los muestra con barras y colores, por lo que es más fácil visualizar el consumo.

  La información proviene del mismo sistema; `htop` no mide otra memoria distinta, sino que la presenta de una manera más clara e interactiva.

`kill`: controlar o finalizar procesos

Primero buscás el **PID** con `ps`, `top` o `htop`. Después:

kill [señal] PID

Machete básico

kill PID

Finaliza el proceso de forma normal. Equivale a:

kill -TERM PID
kill -STOP PID

Suspende o pausa el proceso.

kill -CONT PID

Reactiva un proceso suspendido.

kill -HUP PID

Solicita que el proceso recargue su configuración. En algunos servicios puede provocar un reinicio, pero depende del programa.

kill -KILL PID

Fuerza la finalización inmediata. También puede escribirse:

kill -9 PID

Señal	Acción
`TERM`	terminar normalmente
`STOP`	suspender
`CONT`	continuar
`HUP`	recargar o reiniciar, según el programa
`KILL`	terminar por la fuerza

¿Cuándo se usa `sudo`?

sudo kill -TERM PID

Se usa cuando el proceso pertenece a otro usuario o requiere permisos de administrador. Para tus propios procesos, normalmente no hace falta.

`/proc`

`/proc` es un sistema de archivos virtual que muestra información del sistema y de los procesos en ejecución.

No contiene archivos guardados realmente en el disco: la información la genera el kernel en tiempo real.

Cada proceso tiene una carpeta identificada por su PID:

/proc/PID

Ejemplo:

/proc/2450

Dentro de esa carpeta se puede consultar información del proceso, como:

- `status`: estado general, memoria y cantidad de hilos.
- `cmdline`: comando con el que se inició.
- `exe`: programa ejecutable.
- `cwd`: directorio de trabajo actual.
- `fd`: archivos y recursos abiertos.

Ejemplo:

cat /proc/2450/status

> Cuando el proceso termina, su carpeta dentro de `/proc` desaparece.

strace

`strace` permite ver las **llamadas al sistema** que realiza un proceso, es decir, cómo el programa le solicita servicios al kernel.

Para rastrear un proceso que ya está ejecutándose:

sudo strace -p PID

Ejemplo:

sudo strace -p 2450

Puede mostrar llamadas como:

- `openat`: abrir un archivo.
- `read`: leer datos.
- `write`: escribir datos.
- `close`: cerrar un archivo.
- `mmap`: asignar memoria.
- `futex`: sincronizar hilos.

Para ejecutar un programa y rastrearlo desde el inicio:

strace ls

Importante:

ls /proc/PID

no ejecuta `strace`. Solo muestra la información disponible del proceso dentro de `/proc`.

> `/proc` muestra el estado del proceso; `strace` muestra lo que el proceso le está pidiendo al sistema operativo en tiempo real.

/proc/PID/

`/proc/PID/` es un **directorio**, no un comando.

Por eso, si se escribe directamente:

/proc/2450/

Bash responde:

Is a directory

Para obtener información hay que:

cd /proc/2450

o leer/listar su contenido:

ls /proc/2450
cat /proc/2450/status

`/proc/PID/fd/`

Es un directorio que contiene los **descriptores de archivos abiertos** por el proceso.

ls -l /proc/2450/fd

Muestra enlaces simbólicos hacia archivos, terminales, sockets o recursos que el proceso tiene abiertos.

> `/proc/PID/` contiene información del proceso; no se ejecuta, se explora o se leen sus arch

`watch`

Ejecuta repetidamente un comando y actualiza el resultado en pantalla.

watch -n 1 "ps -e"

- `watch`: repite el comando.
- `-n 1`: establece un intervalo de **1 segundo**.
- `ps -e`: muestra todos los procesos.

> En este caso, vuelve a ejecutar `ps -e` cada segundo, permitiendo observar cómo aparecen o desaparecen procesos casi en tiempo real.

Para salir:

Ctrl + C

`pidstat`

Sirve para monitorear el uso de recursos de uno o varios procesos en intervalos de tiempo.

Instalación

sudo apt install sysstat

Ejemplo con Firefox

pidstat -p $(pgrep -u usuario firefox) 1

- `pgrep`: busca el PID de Firefox.
- `-u usuario`: limita la búsqueda a ese usuario.
- `-p`: indica qué proceso monitorear.
- `1`: actualiza la información cada 1 segundo.

Estadísticas principales

Por defecto, `pidstat` muestra principalmente uso de CPU.

- **UID:** usuario propietario del proceso.
- **PID:** identificador del proceso.
- **%usr:** CPU usada ejecutando código del programa.
- **%system:** CPU usada ejecutando código del kernel.
- **%guest:** CPU usada para máquinas virtuales.
- **%wait:** tiempo esperando para poder usar la CPU.
- **%CPU:** porcentaje total de CPU consumido.
- **CPU:** núcleo en el que se ejecutó por última vez.
- **Command:** nombre del proceso.

`free`

Muestra el uso de la memoria RAM y de la memoria swap del sistema.

Lo más cómodo es usar:

free -h

`-h` muestra los valores en formato legible, por ejemplo MB o GB.

Columnas

- **total:** memoria total disponible.
- **used:** memoria utilizada.
- **free:** memoria completamente libre.
- **shared:** memoria compartida, por ejemplo entre procesos.
- **buff/cache:** memoria usada por el sistema como búfer y caché.

- **available:** memoria que realmente puede usarse para abrir nuevos programas sin recurrir a swap.

Filas

- **Mem:** información de la memoria RAM.
- **Swap:** información de la memoria de intercambio en disco.

Ejemplo:

```
               total        used        free      shared  buff/cache   available
Mem:            16Gi        6Gi        2Gi       500Mi        8Gi        9Gi
Swap:          2Gi          0Gi        2Gi
```

Comparación con `top` y `htop`

Los tres muestran información de memoria, pero:

- `free` muestra un resumen general de RAM y swap.
- `top` muestra memoria general y además el consumo de cada proceso.
- `htop` muestra lo mismo de forma más visual e interactiva.

Después de eliminar un proceso

Al finalizar un proceso y ejecutar nuevamente:

free -h

puede ocurrir que:

- disminuya `used`;
- aumente `free` o `available`;
- parte de la memoria pase a `buff/cache`.

No siempre el cambio aparece directamente en `free`, porque Linux aprovecha la memoria liberada para caché. Por eso, para saber cuánta memoria queda realmente disponible, conviene observar principalmente la columna **available**.

`smem`

Permite analizar el uso de memoria de cada proceso de forma más precisa que `top` o `ps`, porque diferencia la memoria privada de la memoria compartida.

Instalación

sudo apt install smem

Ejecución

smem -r -k

- `-r`: ordena los procesos de mayor a menor consumo de memoria.
- `-k`: muestra los valores en KB, MB o GB para facilitar la lectura.

Columnas principales

- **PID:** identificador del proceso.
- **User:** usuario propietario del proceso.
- **Command:** comando o programa ejecutado.
- **Swap:** memoria swap usada por el proceso.
- **USS:** memoria física exclusiva del proceso, no compartida con otros.
- **PSS:** memoria del proceso sumando su memoria privada y una parte proporcional de la memoria compartida.
- **RSS:** memoria RAM total asociada al proceso, incluyendo toda la memoria compartida.

Diferencia entre USS, PSS y RSS

- **USS:** lo que se liberaría aproximadamente si se cerrara únicamente ese proceso.
- **PSS:** estimación más realista del consumo de memoria, porque reparte la memoria compartida entre los procesos que la usan.
- **RSS:** puede exagerar el consumo total, porque cuenta completa la memoria compartida para cada proceso.

  `smem -r -k` muestra los procesos ordenados por consumo de memoria y permite distinguir memoria exclusiva, proporcional y total. Para comparar procesos, normalmente la columna más útil es **PSS**.

`vmstat`

Muestra un resumen del estado del sistema: procesos, memoria, swap, entrada/salida, actividad del sistema y CPU.

vmstat

También se puede actualizar cada segundo:

vmstat 1

Columnas de `vmstat`

`procs`

Procesos del sistema.

- **r:** procesos listos para ejecutarse, esperando CPU.
- **b:** procesos bloqueados, normalmente esperando entrada/salida.

`memory`

Uso de memoria.

- **swpd:** memoria swap utilizada.
- **free:** memoria RAM libre.
- **buff:** memoria usada como búfer.
- **cache:** memoria usada como caché.

`swap`

Intercambio entre RAM y disco.

- **si:** memoria que entra desde swap hacia RAM por segundo.
- **so:** memoria que sale desde RAM hacia swap por segundo.

`io`

Actividad de entrada/salida de bloques.

- **bi:** bloques leídos desde dispositivos hacia memoria por segundo.
- **bo:** bloques escritos desde memoria hacia dispositivos por segundo.

`system`

Actividad del kernel.

- **in:** interrupciones por segundo.
- **cs:** cambios de contexto por segundo.

`cpu`

Porcentaje de tiempo de CPU.

- **us:** tiempo ejecutando programas de usuario.
- **sy:** tiempo ejecutando código del kernel.
- **id:** tiempo de CPU inactiva.
- **wa:** tiempo esperando operaciones de entrada/salida.
- **st:** tiempo quitado por el hipervisor en una máquina virtual.

Para recordar

`r` alto indica procesos esperando CPU.
`b` o `wa` altos suelen indicar espera por entrada/salida.

`si` y `so` altos indican uso frecuente de swap.
`id` alto significa que la CPU está mayormente libre.

`/proc/meminfo`

Es un archivo virtual que muestra información detallada y actualizada sobre la memoria del sistema.

cat /proc/meminfo

Datos principales:

- **MemTotal:** memoria RAM total.
- **MemFree:** RAM completamente libre.
- **MemAvailable:** memoria disponible para nuevos procesos sin usar swap.
- **Buffers:** memoria usada como búfer.
- **Cached:** memoria usada como caché.
- **SwapTotal:** memoria swap total.
- **SwapFree:** memoria swap libre.
- **Active:** memoria usada recientemente.
- **Inactive:** memoria poco utilizada, que puede recuperarse.
- **Dirty:** memoria modificada que todavía debe escribirse al disco.
- **PageTables:** memoria utilizada por las tablas de páginas.

`/proc/meminfo` ofrece información más detallada que `free`, mientras que `free` toma esos datos y los presenta resumidos y de manera más legible.

Sistemas de archivos montados

Hay tres formas comunes de verlos:

`cat /proc/mounts`

cat /proc/mounts

Muestra los sistemas de archivos que el kernel tiene montados actualmente.

Columnas principales:

- **dispositivo**: partición o recurso montado.
- **punto de montaje**: carpeta donde está montado.
- **tipo de sistema de archivos**: por ejemplo `ext4`, `tmpfs`, `proc`.
- **opciones**: permisos y forma de montaje, como `rw` o `ro`.

`mount`

mount

También muestra los sistemas de archivos montados, con un formato más legible.

`findmnt`

findmnt

Muestra los montajes en forma de árbol y suele ser el más claro.

Columnas comunes:

- **TARGET**: punto de montaje.
- **SOURCE**: dispositivo o recurso de origen.
- **FSTYPE**: tipo de sistema de archivos.
- **OPTIONS**: opciones de montaje.

> Los tres muestran información similar; `findmnt` suele ser el más ordenado y fácil de interpretar.

`/etc/fstab`

El archivo `/etc/fstab` contiene la configuración de los sistemas de archivos que deben montarse automáticamente al iniciar Linux.

cat /etc/fstab

Cada línea suele tener estas columnas:

- **Dispositivo o UUID:** identifica la partición o disco.
- **Punto de montaje:** carpeta donde se monta, por ejemplo `/`, `/home` o `/mnt/datos`.
- **Tipo de sistema de archivos:** por ejemplo `ext4`, `ntfs`, `vfat`, `swap`.
- **Opciones de montaje:** por ejemplo `defaults`, `ro`, `rw`, `noauto`.
- **Dump:** indica si se incluye en copias de seguridad con `dump`.
- **Pass:** orden en que se revisa el sistema de archivos con `fsck`.

Ejemplo:

UUID=xxxx-xxxx /home ext4 defaults 0 2

Significa que esa partición:

- se identifica por su UUID,
- se monta en `/home`,
- usa `ext4`,
- se monta con opciones normales,
- y se revisa después de la partición raíz.

> `fstab` no muestra solo lo que está montado ahora, sino lo que está configurado para montarse automáticamente.

`lsblk -f`

Muestra los **dispositivos de bloque** —discos y particiones— junto con información de sus sistemas de archivos.

lsblk -f

Columnas principales:

- **NAME:** nombre del disco o partición.
- **FSTYPE:** tipo de sistema de archivos.
- **FSVER:** versión del sistema de archivos.
- **LABEL:** nombre o etiqueta asignada.
- **UUID:** identificador único.
- **FSAVAIL:** espacio disponible.
- **FSUSE%:** porcentaje de espacio utilizado.
- **MOUNTPOINTS:** directorio donde está montado.

En **FSTYPE** se pueden identificar, por ejemplo:

- `ext4`: sistema común de Linux.
- `vfat` o `fat32`: común en pendrives y particiones EFI.
- `ntfs`: usado principalmente por Windows.
- `exfat`: frecuente en discos extraíbles.
- `swap`: área de intercambio de Linux.
- `xfs`, `btrfs`: otros sistemas usados en Linux.

> `lsblk -f` permite identificar qué discos y particiones existen, qué sistema de archivos usa cada uno y dónde están montados.

Crear un archivo de imagen con `dd`

dd if=/dev/zero of=mi_imagen.img bs=1M count=100

- **`dd`**: copia datos byte a byte desde un origen hacia un destino.
- **`if=/dev/zero`**: archivo de entrada; `/dev/zero` genera ceros continuamente.
- **`of=mi_imagen.img`**: archivo de salida que se va a crear.
- **`bs=1M`**: tamaño de cada bloque: 1 MB.
- **`count=100`**: copia 100 bloques.

Resultado:

> Crea un archivo de aproximadamente **100 MB lleno de ceros**.

Importante:

Todavía no es un sistema de archivos ni un disco montable. Es solo un archivo vacío a nivel lógico, que después puede formatearse y usarse como una imagen de disco.

Formatear la imagen

mkfs.ext4 mi_imagen.img

- **`mkfs`**: crea un sistema de archivos.
- **`ext4`**: tipo de sistema de archivos utilizado.
- **`mi_imagen.img`**: archivo donde se crea ese sistema de archivos.

Se utilizó el sistema de archivos **ext4**. Ahora el archivo puede tratarse como una imagen de disco formateada.

Montar la imagen

sudo mkdir /mnt/mi_imagen
sudo mount -o loop mi_imagen.img /mnt/mi_imagen

- **`mkdir`**: crea el punto de montaje.
- **`mount`**: monta el sistema de archivos.
- **`-o loop`**: hace que Linux trate el archivo `.img` como si fuera un dispositivo de bloque.
- **`/mnt/mi_imagen`**: carpeta donde se podrá acceder a su contenido.

Verificar el montaje

df -h | grep mi_imagen

- **`df -h`**: muestra espacio total, usado y disponible de los sistemas montados.
- **`grep mi_imagen`**: filtra únicamente la línea correspondiente a la imagen.

Resultado: `mi_imagen.img`, formateada como `ext4`, queda accesible desde `/mnt/mi_imagen`.

Desmontar la imagen

sudo umount /mnt/mi_imagen

- **`umount`**: desmonta el sistema de archivos.
- Después de desmontarlo, el contenido deja de estar accesible desde `/mnt/mi_imagen`.

Agregarla a `/etc/fstab`

Ejemplo:

echo "$(pwd)/mi_imagen.img /mnt/mi_imagen ext4 loop 0 0" | sudo tee -a /etc/fstab

Campos agregados:

- **`$(pwd)/mi_imagen.img`**: ruta completa del archivo de imagen.
- **`/mnt/mi_imagen`**: punto de montaje.
- **`ext4`**: tipo de sistema de archivos.
- **`loop`**: trata el archivo como un dispositivo de bloque.
- **primer `0`**: no usar `dump`.
- **segundo `0`**: no comprobarlo automáticamente con `fsck` durante el arranque.
- **`tee -a`**: agrega la línea al final de `/etc/fstab` con permisos de administrador.

Probar la configuración

sudo mount -a

Monta todos los sistemas de archivos configurados en `/etc/fstab` que todavía no estén montados.

Verificación:

df -h | grep mi_imagen

> `/etc/fstab` guarda la configuración de montaje automático. `loop` permite montar un archivo como si fuera un disco y `mount -a` prueba o aplica todas las entradas del archivo.

Importante: antes de editar `/etc/fstab`, conviene hacer una copia:

sudo cp /etc/fstab /etc/fstab.bak

Enlace simbólico y enlace duro

Enlace simbólico

ln -s original.txt enlace_simbolico.txt

Es un archivo especial que **apunta al nombre o ruta del archivo original**.

- Tiene un inodo distinto.
- Puede apuntar a archivos o directorios.
- Puede apuntar a otro sistema de archivos.
- Si se borra el original, el enlace queda roto.

Enlace duro

ln original.txt enlace_duro.txt

Es otro nombre para el **mismo contenido físico**.

- Comparte el mismo inodo que el original.
- No depende del nombre original.
- Normalmente no se usa para directorios.
- No puede cruzar sistemas de archivos.

Al modificar el archivo original

Si cambiás `original.txt`, el cambio se ve desde:

- `enlace_simbolico.txt`
- `enlace_duro.txt`

Ambos permiten acceder al mismo contenido mientras el original existe.

Al borrar el archivo original

rm original.txt

- **Enlace simbólico:** queda roto, porque apuntaba a la ruta eliminada.
- **Enlace duro:** sigue funcionando, porque todavía existe otro nombre asociado al mismo inodo y contenido.

Comparación rápida

Característica	Simbólico	Duro
Apunta a	Ruta del archivo	Mismo inodo
Inodo	Distinto	Igual
Si se borra el original	Se rompe	Sigue funcionando
Puede cruzar discos	Sí	No
Puede enlazar directorios	Sí	Generalmente no

> El enlace simbólico es un acceso directo. El enlace duro es otro nombre para el mismo archivo.

Glosario básico de comandos Linux

Estos son pocos, pero sirven para entender casi todos los ejercicios que hiciste:

- **`cat archivo`**: muestra el contenido de un archivo en la terminal. No lo abre en un editor; solamente lo imprime.

cat /etc/fstab

- **`ls`**: lista archivos y directorios.

ls
ls -l

- **`cd directorio`**: cambia de directorio.

cd /proc

- **`pwd`**: muestra la ruta del directorio actual.

pwd

- **`mkdir nombre`**: crea un directorio.

mkdir prueba

- **`cp origen destino`**: copia archivos o directorios.

cp archivo.txt copia.txt

- **`mv origen destino`**: mueve o renombra archivos.

mv viejo.txt nuevo.txt

- **`rm archivo`**: elimina un archivo.

rm archivo.txt

- **`grep texto archivo`**: busca líneas que contengan un texto.

grep ext4 /etc/fstab

También puede filtrar la salida de otro comando:

df -h | grep mi_imagen

- **`sudo comando`**: ejecuta un comando con permisos de administrador.

sudo mount -a

- **`man comando`**: muestra el manual de un comando.

man ps

Símbolos muy usados

- `|`: envía la salida de un comando al siguiente.
- `>`: guarda la salida reemplazando el archivo.
- `>>`: agrega la salida al final del archivo.
- `-`: introduce opciones, como `-h`, `-f` o `-r`.

`cat` muestra contenido; `ls` lista; `cd` entra; `pwd` ubica; `grep` busca; `sudo` da permisos; `man` explica comandos.